MINI PROJECT - PART 1

Unit testing the core business logic of the Hotel Booking solution

Software Quality

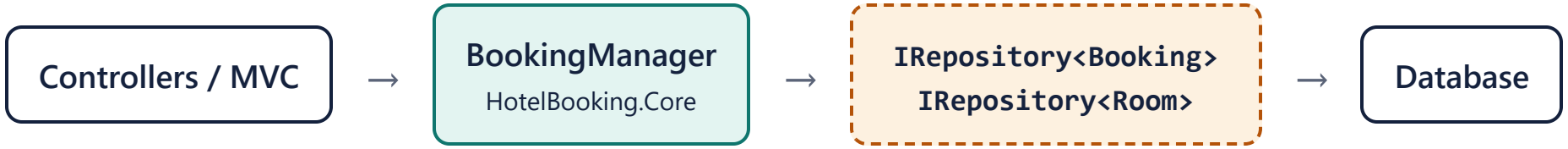
OVERVIEW

What we will show in 15 minutes

1. **System under test** - what `BookingManager` does and how it is wired
2. **Test design** - structure, naming, shared setup
3. **Mocking** - isolating the logic from the database with Moq
4. **Data-driven test** - boundary value analysis with `[Theory]`
5. **Bugs the tests found** - and how we fixed them
6. **Coverage** of the business logic
7. **SonarCloud** and the **Project health dashboard**
8. **Conclusions**

SYSTEM UNDER TEST

BookingManager holds the business rules



Repositories are injected through the constructor, so tests can replace them with fakes.

METHOD	RULE TO TEST
`FindAvailableRoom`	Start date must be in the future and not after the end date. Returns the first room without an overlapping *active* booking, or -1.
`CreateBooking`	If a room is free: assign it, mark the booking active, save it. Otherwise return false and save nothing.
`GetFullyOccupiedDates`	Return every date in the period on which *all* rooms have an active booking.

TEST DESIGN

32 unit tests, one consistent structure

- **xUnit v3** as test framework, **Moq** as mocking framework
- **Arrange - Act - Assert** in every test
- Names read as a sentence: `Method_Scenario_ExpectedResult`
- Constructor creates the mocks and a default hotel with **2 rooms**; each test only sets up what it needs
- Helper `SetUpBookings(...)` keeps the Arrange part short
- Dates are relative to `DateTime.Today`, so tests never expire

How the cases were chosen

Equivalence classes: invalid dates, no bookings, free room, all rooms taken, inactive (cancelled) booking.

Boundary values: the day before, the same day, and the day after an existing booking.

```
public BookingManagerTests()
{
    rooms = new List<Room> { /* Id 1 and 2 */ };
    mockBookingRepository = new Mock<IRepository<Booking>>();
    mockRoomRepository    = new Mock<IRepository<Room>>();
    mockRoomRepository.Setup(r => r.GetAllAsync())
                      .ReturnsAsync(rooms);
    bookingManager = new BookingManager(
        mockBookingRepository.Object,
        mockRoomRepository.Object);
}
```

MOCKING FRAMEWORK

Moq replaces the database, so the tests test only the logic

Stubbing - control the input

`Setup(...).ReturnsAsync(...)` decides which bookings and rooms the manager sees. No database, fast and deterministic.

Verification - check the interaction

`Verify(..., Times.Once)` and `Times.Never` prove that `AddAsync` is called when a room is free, and not called when the hotel is full.

```
// CreateBooking: room is free
SetUpBookings();
var booking = new Booking {
    StartDate = Today.AddDays(1),
    EndDate   = Today.AddDays(2) };

await bookingManager.CreateBooking(booking);

mockBookingRepository.Verify(
    r => r.AddAsync(booking), Times.Once);

// CreateBooking: all rooms taken
mockBookingRepository.Verify(
    r => r.AddAsync(It.IsAny<Booking>()),
    Times.Never);
```

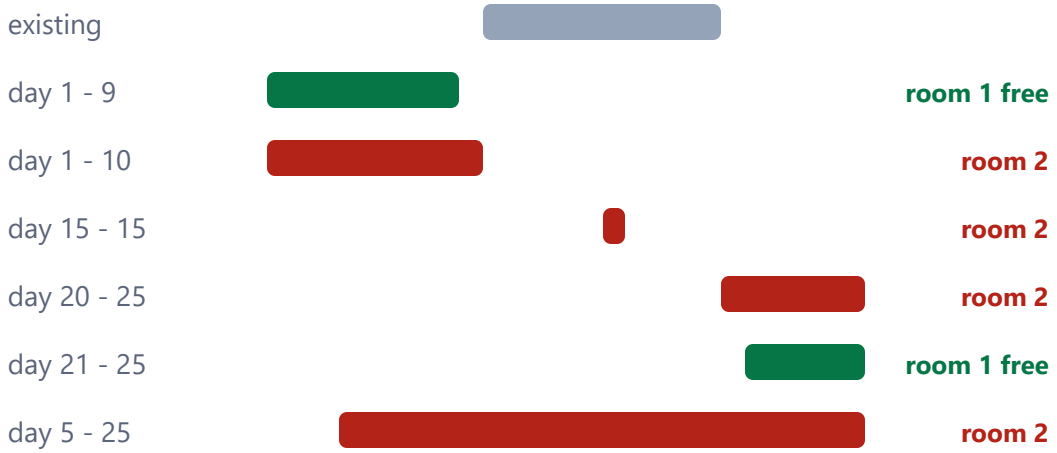
DATA-DRIVEN UNIT TEST

One test method, six boundary cases

```
public static IEnumerable<object[]> OverlapBoundaryCases()
{
    // startOffset, endOffset, expectRoom1Available
    yield return new object[] {  1,  9, true  };
    yield return new object[] {  1, 10, false };
    yield return new object[] { 15, 15, false };
    yield return new object[] { 20, 25, false };
    yield return new object[] { 21, 25, true  };
    yield return new object[] {  5, 25, false };
}

[Theory]
[MemberData(nameof(OverlapBoundaryCases))]
public async Task FindAvailableRoom_
    OverlapWithExistingBooking_ReturnsExpectedAvailability(
    int startOffset, int endOffset, bool expectRoom1)
{
    // existing booking: room 1, day 10 to day 20
    ...
}
```

Existing booking on room 1: day 10 to day 20



Red = overlaps, so room 1 is skipped. Green = no overlap. The interesting cases sit exactly on the edges (day 10 and day 20), where `<` versus `<=` matters.

DATA-DRIVEN UNIT TEST

A second theory: which dates are fully occupied?

`GetFullyOccupiedDates` is tested with a `[MemberData]` theory whose data includes whole arrays of bookings and the expected dates.

CASE	EXPECTED RESULT
Both rooms booked for the whole period	every date returned
Both rooms booked for the middle day only	only that day
One booking is inactive (cancelled)	empty list

```
[Theory]
[MemberData(nameof(FullyOccupiedDatesCases))]
public async Task GetFullyOccupiedDates_
    VariousBookingConfigurations_ReturnsExpectedDates(
    Booking[] bookings, DateTime start,
    DateTime end, DateTime[] expectedDates)
{
    // Arrange
    SetUpBookings(bookings);

    // Act
    List<DateTime> occupied =
        await bookingManager
            .GetFullyOccupiedDates(start, end);

    // Assert
    Assert.Equal(expectedDates, occupied);
}
```

BUGS FOUND BY THE TESTS

Bug 1: bookings were counted instead of rooms

`GetFullyOccupiedDates` compared the number of **bookings** on a date with the number of rooms.

> **Example:** 2 rooms, and two active bookings on *room 1* for the same days. The code counts 2 bookings, 2 ≥ 2, and reports the dates as fully occupied. Room 2 is actually free.

Regression test

```
// both bookings on room 1, room 2 is free
SetUpBookings(
  new Booking { RoomId = 1, IsActive = true, ... },
  new Booking { RoomId = 1, IsActive = true, ... });

var result = await bookingManager
    .GetFullyOccupiedDates(start, end);

Assert.Empty(result);
```

Step 1 - the test fails on the old code RED

The method returned every date from day 5 to day 10 instead of an empty list.

Step 2 - the fix GREEN

```
// OLD (bug)
// if (noOfBookings.Count() >= noOfRooms)

// FIXED: count each booked room once
if (noOfBookings
      .Select(b => b.RoomId)
      .Distinct()
      .Count() >= noOfRooms)
    fullyOccupiedDates.Add(d);
```

After the fix all 32 tests pass.

Bug 2: off-by-one in the overlap check

In `FindAvailableRoom` a comparison used `<=` where it needed `<`.

```
// wrong: request ending ON the existing
// start date looked like "no overlap"
startDate <= b.StartDate && endDate <= b.StartDate

// correct
startDate < b.StartDate && endDate < b.StartDate
```

Why the data-driven test catches it

The case `{ 1, 10, false }` asks for a stay that ends exactly on the day the existing booking starts. With `<=` the room is wrongly returned as free, so the assertion fails.

A single "typical" test would never hit this edge. This is why boundary values are worth a theory.

COVERAGE

The business logic is fully covered

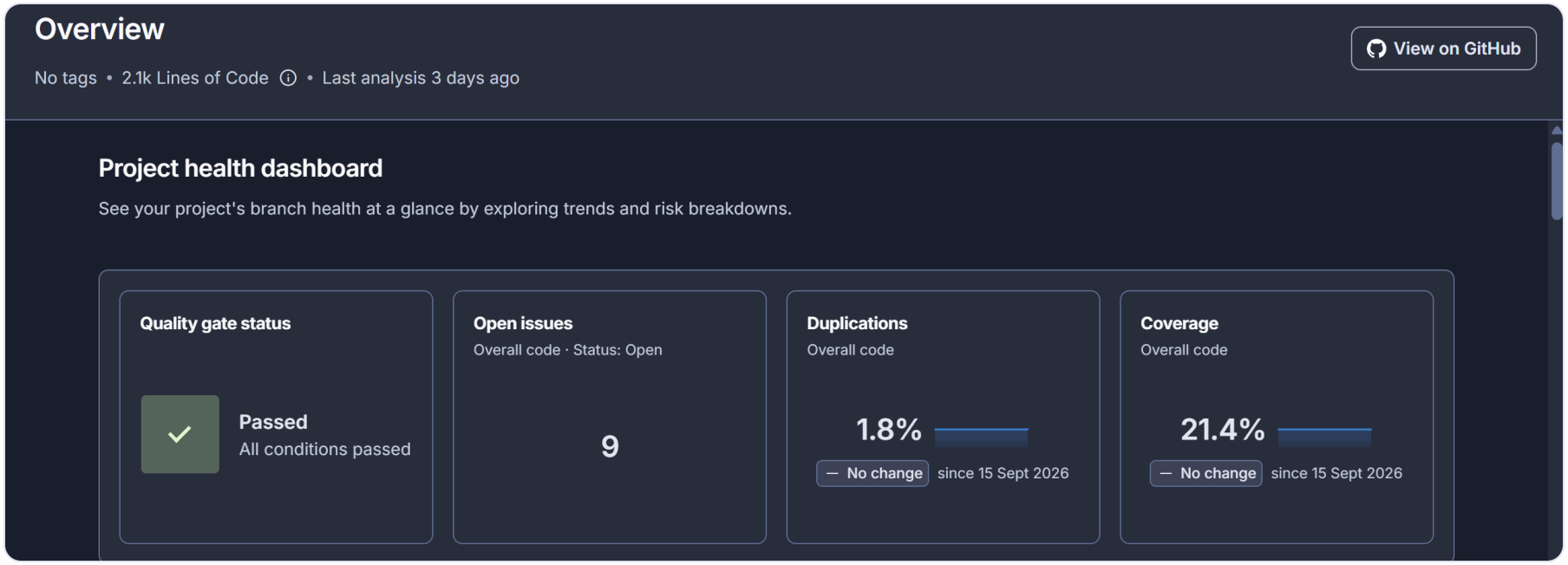
100%

line and branch coverage of `BookingManager`

measured locally with `dotnet-coverage`, 32 tests, 0 failures

- All three public methods and every branch of the validation and overlap conditions are exercised.
- Coverage is a floor, not proof: the bugs on the previous slides were found by **choosing the right inputs** (same room twice, edge dates), not by the coverage number.
- Inactive bookings and a hotel with three rooms are tested explicitly, so the room count is not hard-coded to 2.

The Project health dashboard on SonarCloud



Quality gate: Passed

All conditions pass on the latest analysis.

9 open issues, 1.8% duplication

4 reliability issues (all medium, rating C) and 5 maintainability issues (rating A). Duplication is low.

Coverage 21.4% (whole solution)

Counts MVC, Web API and Infrastructure too, which are outside the scope of the Core unit tests. On the SonarCloud Code tab, **BookingManager.cs itself shows 100%**, 0 uncovered lines.

CONCLUSIONS

What we learned

- **Dependency injection** is what makes the code testable: repositories are interfaces, so Moq can replace them.
- **Mocks** give speed and control; `Verify` checks behaviour, not only return values.
- **Data-driven tests** turn boundary analysis into a compact table of cases.
- Tests found **two real bugs**: the wrong count and the off-by-one comparison.
- Write the failing test first, then fix: red, green, and the test stays as a regression guard.
- Static analysis and coverage tell us *where to look*; well-chosen inputs tell us *if it is right*.

Thank you - questions?